

MINI-PROJET

Optimisation stratégique de la couverture Wi-Fi
au sein d'une infrastructure d'entreprise

Réalisé par : Zeroual Hicham

P lan du mini-projet

I. Introduction

II. Objectifs du projet

III. Méthodologie

IV. Comparaison entre Cas 1 et Cas 2

V. Conclusion

VI. Résumé / Summary / Zusammenfassung

I. Introduction

✓ Contexte du besoin en connectivité Wi-Fi dans les entreprises

Dans un monde de plus en plus numérique, la connectivité Wi-Fi est devenue un élément fondamental pour le bon fonctionnement des entreprises. Elle permet aux employés d'accéder rapidement aux ressources partagées, de communiquer efficacement, d'utiliser des outils collaboratifs en ligne, et de garantir la continuité des activités. Une bonne infrastructure réseau est donc essentielle pour améliorer la productivité et l'expérience utilisateur au sein de l'entreprise.

⚠ Problématique rencontrée : couverture incomplète, instabilité réseau

Cependant, dans de nombreuses structures, des zones dites "mortes" ou mal couvertes persistent, causant une mauvaise qualité de service. Ces problèmes de couverture Wi-Fi peuvent entraîner :

- des coupures de connexion fréquentes
- une latence élevée
- une insatisfaction des utilisateurs

Ces défauts sont généralement dus à un mauvais positionnement des points d'accès Wi-Fi (APs), à une mauvaise planification du réseau, ou à l'absence de prise en compte des obstacles physiques (murs, meubles, etc.).

🎯 Objectif global du projet

L'objectif principal de ce mini-projet est de proposer une solution d'optimisation de la couverture Wi-Fi, en utilisant une approche algorithmique pour déterminer le meilleur positionnement des points d'accès dans une salle donnée. Cette solution vise à :

- maximiser la couverture du réseau
- réduire les zones non couvertes
- améliorer la stabilité du signal de manière économique.

II. Objectifs du projet

L'objectif **principal** de ce mini-projet est d'optimiser la couverture Wi-Fi dans un environnement d'entreprise en identifiant les zones mal desservies et en proposant un déploiement stratégique des points d'accès. Le projet s'inscrit dans une démarche d'amélioration de la connectivité, de la qualité du service et de la satisfaction des utilisateurs.

Les objectifs spécifiques sont :

- 🔍 Analyser l'état actuel de la couverture Wi-Fi dans l'entreprise afin d'identifier les zones non couvertes ou mal desservies.
- ❖ Utiliser des outils de cartographie comme QGIS pour modéliser l'environnement et localiser précisément les zones problématiques.
- 📁 Développer un script Python pour automatiser le placement optimal des points d'accès Wi-Fi, en tenant compte de paramètres comme la portée du signal et les obstacles.
- ✓ Tester et valider la solution proposée à l'aide de simulations, afin de garantir une couverture optimale et stable dans toute la zone concernée.
- 📊 Évaluer les performances du réseau optimisé et proposer des recommandations pour un déploiement réel

III. Méthodologie

Pour atteindre l'objectif d'améliorer la couverture Wi-Fi, nous avons suivi une série d'étapes organisées et structurées comme suit :

❖ Modélisation de la salle

Conception d'un plan 2D de l'entreprise

dans cette phase, nous avons conçu un plan bidimensionnel (2D) représentant les différents espaces de l'entreprise, tels que l'accueil, la direction, les ressources humaines, l'open space, la salle de réunion, la salle des serveurs, le stock, ainsi que les autres installations.

🎯 **Objectif principal** : comprendre la répartition des espaces et identifier les zones qui nécessitent une couverture Wi-Fi optimale. Cela permet d'élaborer ultérieurement une stratégie efficace de déploiement des points d'accès sans fil (APs).

✓ **Résultat** : une visualisation précise et claire de l'espace de travail, servant de base solide pour les étapes suivantes de la planification réseau.

❖ Code pour dessiner le plan 2D de l'entreprise

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# Création de la figure
fig, ax = plt.subplots(figsize=(15, 9))
ax.set_xlim(0, 50)
ax.set_ylim(0, 30)
ax.set_title("Plan de l'entreprise (50m x 30m)", fontsize=16)

# Liste des pièces avec [nom, x, y, largeur, hauteur, couleur]
rooms = [
    ("Zone d'accueil", 0, 0, 10, 5, '#FFDDC1'),
    ("Bureau Direction", 0, 5, 6, 6, '#FFD700'),
    ("Bureau RH", 6, 5, 6, 5, '#F08080'),
    ("Open Space", 12, 5, 20, 10, '#87CEEB'),
    ("Salle Réunion", 32, 5, 10, 6, '#90EE90'),
    ("Salle Serveur", 42, 5, 4, 3, '#D3D3D3'),
    ("Sanitaires + Cuisine", 0, 11, 8, 5, '#FFB6C1'),
]
```

```
# Ajouter les rectangles des pièces
for name, x, y, w, h, color in rooms:
    rect = Rectangle((x, y), w, h, linewidth=1.5, edgecolor='black', facecolor=color)
    ax.add_patch(rect)
    ax.text(x + w/2, y + h/2, name, ha='center', va='center', fontsize=9, wrap=True)

# Bordure du bâtiment
ax.plot([0, 50, 50, 0, 0], [0, 0, 30, 30, 0], 'black', linewidth=2)

# Options d'affichage
ax.set_aspect('equal')
ax.set_xlabel("Largeur (m)")
ax.set_ylabel("Longueur (m)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

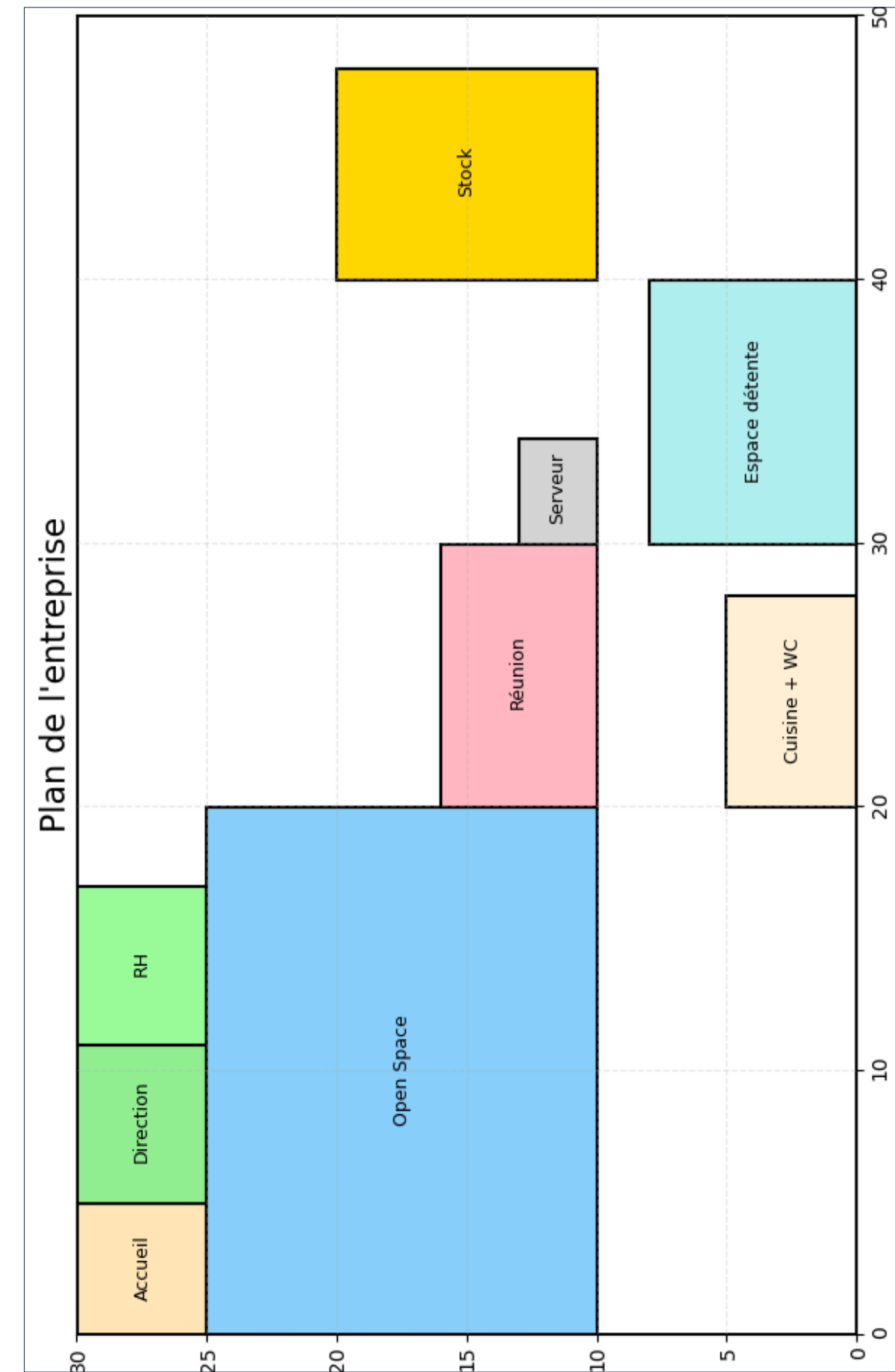

❖ Utilisation de la bibliothèque matplotlib

J'ai utilisé la bibliothèque matplotlib avec le module patches pour créer une carte en deux dimensions représentant l'espace de l'entreprise. J'ai d'abord défini les dimensions globales de la carte (largeur et hauteur), puis ajouté des rectangles colorés pour représenter les différentes zones (accueil, bureaux, salle de réunion, etc.). Chaque zone a été positionnée précisément avec ses dimensions et une étiquette placée au centre. Enfin, un contour a été tracé autour de la surface totale de l'entreprise et une grille a été ajoutée pour faciliter la lecture de la carte.

❖ Objectif de l'utilisation de matplotlib

Nous utilisons la bibliothèque matplotlib pour dessiner une carte claire et simple de l'entreprise en 2D. Ce dessin aide à visualiser la répartition des différentes zones (bureaux, salles, espaces ouverts, etc.) facilement, et nous permet de mieux comprendre l'espace afin de planifier l'amélioration de la couverture Wi-Fi.

Le plan 2D de l'entreprise



❖ Création d'une grille de simulation

- Description de l'étape

Dans cette étape, nous avons divisé la surface de l'entreprise en une grille composée de petits points régulièrement espacés. Chaque point représente une position potentielle où la couverture Wi-Fi peut être évaluée. Cette méthode permet d'analyser la qualité de la couverture de manière précise et détaillée sur l'ensemble de la zone étudiée.

- Code pour créer la grille python

```
import numpy as np

# Dimensions de la salle
largeur_total = 50
hauteur_total = 30

# Résolution de la grille (distance entre points)
pas = 2 # par exemple, points tous les 2 mètres

# Création des coordonnées X et Y
x_points = np.arange(0, largeur_total + pas, pas)
y_points = np.arange(0, hauteur_total + pas, pas)

# Création de la grille de points (tous les couples (x,y))
grille_points = np.array([[x, y] for x in x_points for y in y_points])

print(f"Nombre total de points dans la grille: {len(grille_points)}")
```

Nombre total de points dans la grille: 416

- Utilité de la grille et nombre de points

La grille permet de découper la surface en points précis pour analyser la couverture Wi-Fi en détail. Le nombre total de points (416) reflète la précision de cette division : plus il y a de points, plus l'évaluation est fine et complète.

- Calcul du nombre de points :

Largeur = 50 mètres

Nombre de points sur l'axe horizontal :

$50/2+1=25+1=26$ points

Hauteur = 30 mètres

Nombre de points sur l'axe vertical :

$30/2+1=15+1=16$ points

Nombre total de points dans la grille :

$26 \times 16 = 416$ points

❖ Règle mathématique pour la création d'une grille régulière

Soit une surface rectangulaire de dimensions L (longueur) et W (largeur).

Si on choisit un pas p (distance entre deux points adjacents), alors :

- Le nombre de points le long de la longueur est :
 $NL = [p/L] + 1$
 - Le nombre de points le long de la largeur est :
 $NW = [p/W] + 1$
 - Le nombre total de points dans la grille est :
 $N_{total} = NL \times NW$
- Où $[\cdot]$ est la fonction partie entière (floor).

❖ Importance du nombre de points dans la précision de la simulation Wi-Fi

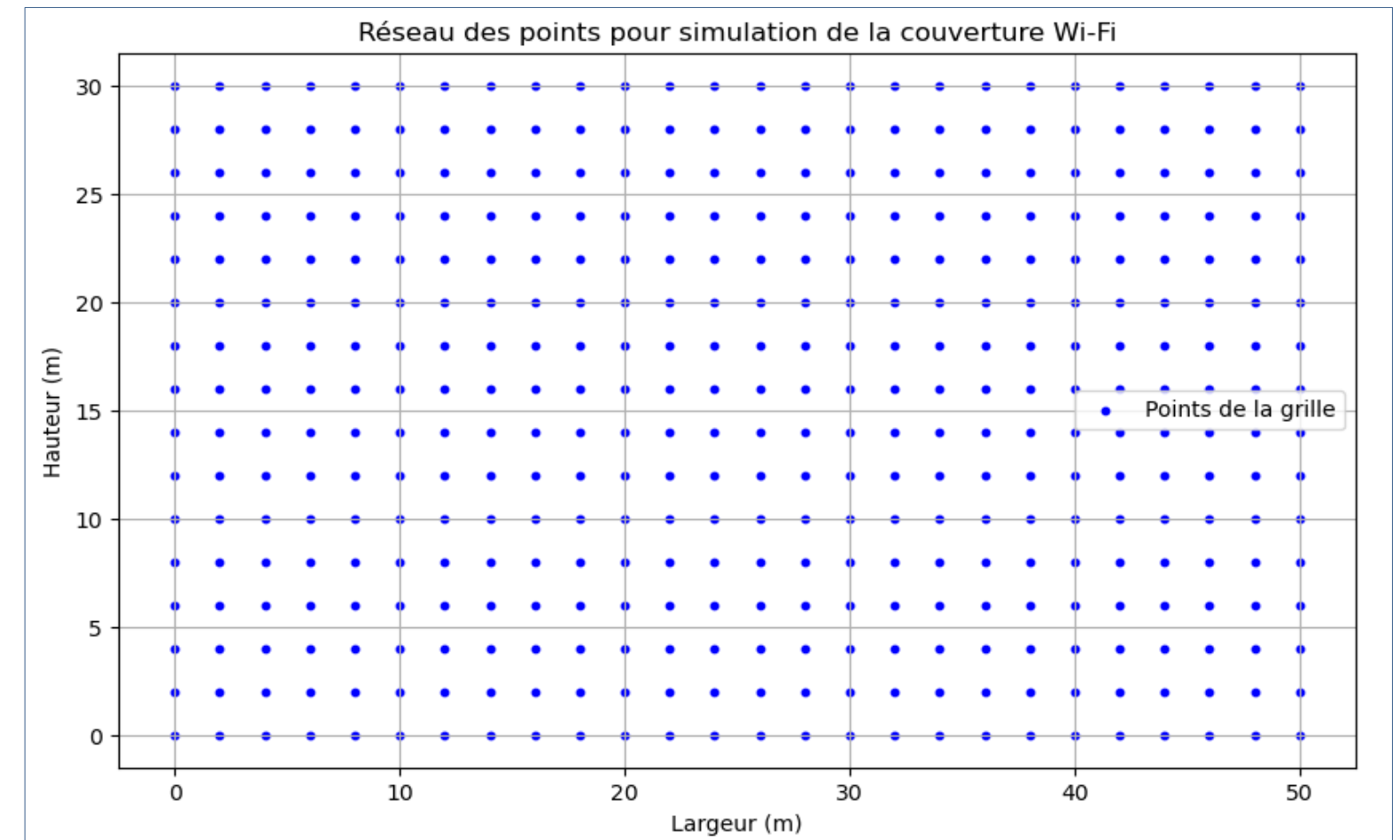
Le total de 416 points correspond à un maillage précis de la surface à étudier. Ce découpage permet une analyse fine et complète de la couverture Wi-Fi. Plus le nombre de points est élevé, plus la simulation est précise.

❖ code de "Visualisation de la grille de points pour l'analyse de couverture Wi-Fi"

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.scatter(grille_points[:,0], grille_points[:,1], c='blue', s=10, label='Points de la grille')
plt.title("Réseau des points pour simulation de la couverture Wi-Fi")
plt.xlabel("Largeur (m)")
plt.ylabel("Hauteur (m)")
plt.grid(True)
plt.legend()
plt.show()
```

"Visualisation de la grille de points pour l'analyse de couverture Wi-Fi"



❖ Calcul couverture initiale

➤ Objectif de l'étape :

L'objectif de cette étape est d'évaluer la situation actuelle du réseau Wi-Fi dans l'entreprise.

➤ Méthodologie utilisée

- 1 . Langage utilisé : Python
- 2 . Bibliothèques : numpy, matplotlib, scipy
- 3 . Création d'une fonction personnalisée : `calcul_taux_couverture()`
- 4 . Création d'une grille de 1m x 1m pour modéliser la salle (50m x 30m)
- 5 . Rayon de couverture de chaque AP : 10m

➤ Déroulement technique

Quatre points d'accès (Access Points) ont été placés manuellement aux coins de la salle, en suivant une répartition classique et simple. Ensuite, la fonction `calcul_taux_couverture()` a été utilisée pour calculer le taux de couverture, c'est-à-dire le pourcentage de points à l'intérieur de la salle qui sont couverts par le signal, en fonction de la distance au point d'accès le plus proche. Aucune optimisation n'a été appliquée à ce stade. Il s'agit de la situation initiale.

❖ Code Python : Calcul de la couverture Wi-Fi

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter

# Dimensions de l'entreprise (en mètres)
longueur = 20
largeur = 10
resolution = 0.1 # taille d'une cellule (précision)

# Grille de points
x = np.arange(0, longueur, resolution)
y = np.arange(0, largeur, resolution)
X, Y = np.meshgrid(x, y)

# Emplacements manuels des Access Points (AP)
APs = [
    (1, 1),
    (1, 9),
    (19, 1),
    (19, 9)
]

# Rayon de couverture efficace (en mètres)
rayon = 5

# Fonction pour calculer la couverture totale
def calcul_couverture(APs, rayon, X, Y):
    couverture = np.zeros_like(X)
    for (x_ap, y_ap) in APs:
        distance = np.sqrt((X - x_ap)**2 + (Y - y_ap)**2)
        couverture += (distance <= rayon).astype(int)
```

```
# Calcul de la couverture
couverture = calcul_couverture(APs, rayon, X, Y)
couverture_flou = gaussian_filter(couverture, sigma=2)

# Calcul du taux de couverture (au moins 1 AP)
nb_points_couverts = np.sum(couverture >= 1)
nb_total_points = couverture.size
taux_couverture = (nb_points_couverts / nb_total_points) * 100

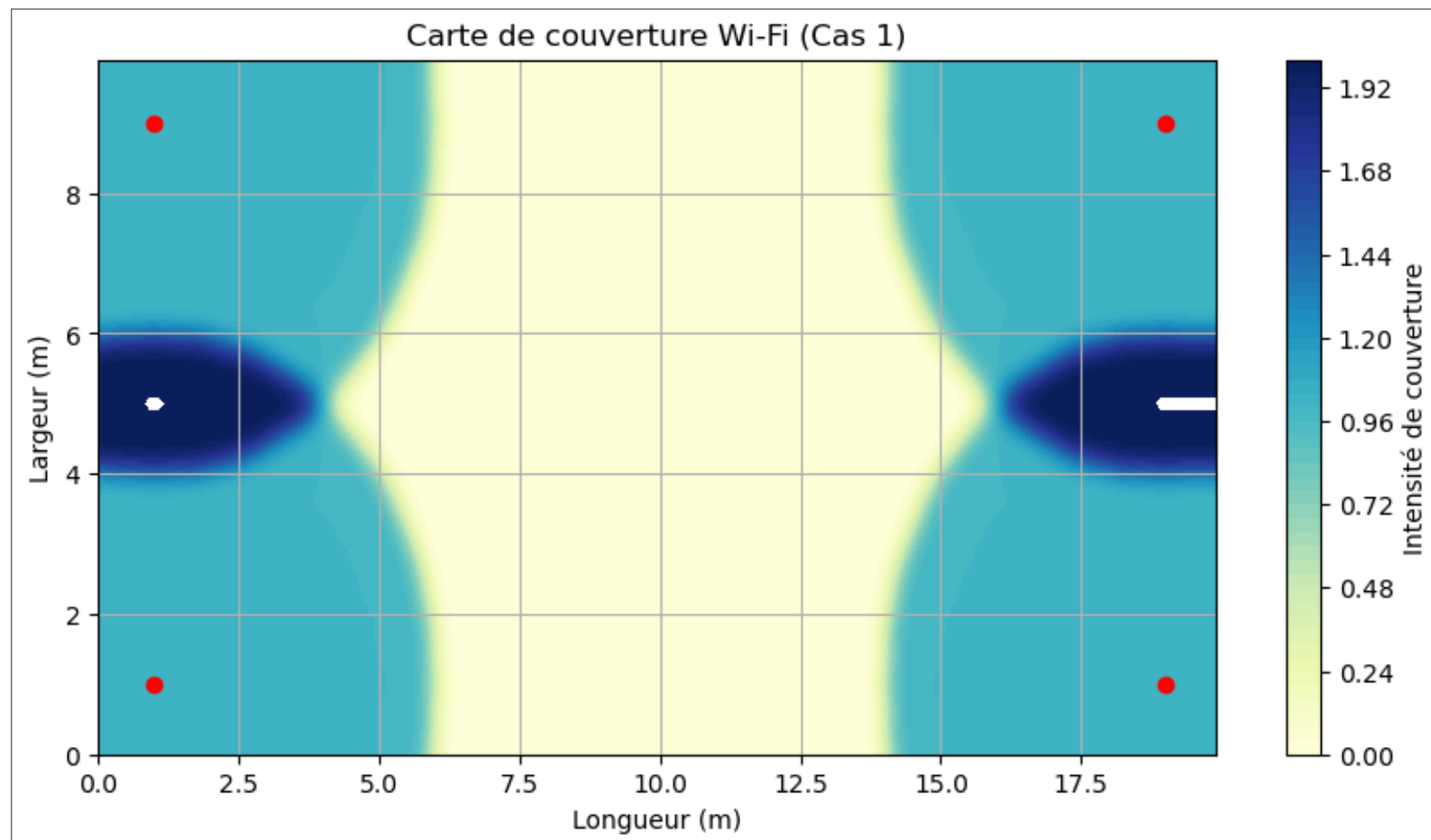
print(f"Taux de couverture : {taux_couverture:.2f}%")

# Affichage de la carte de couverture
plt.figure(figsize=(10, 5))
plt.contourf(X, Y, couverture_flou, levels=50, cmap='YlGnBu')
plt.colorbar(label='Intensité de couverture')
for (x_ap, y_ap) in APs:
    plt.plot(x_ap, y_ap, 'ro') # position des AP
plt.title('Carte de couverture Wi-Fi (Cas 1)')
plt.xlabel('Longueur (m)')
plt.ylabel('Largeur (m)')
plt.grid(True)
plt.show()
```

❖ Résultat initial :

Avant toute optimisation, quatre points d'accès ont été placés aux coins de la salle. L'exécution de la fonction `calcul_taux_couverture()` a permis d'obtenir un taux de couverture initial de **55,05 %**. Cela signifie qu'environ la moitié de la salle seulement est couverte par le signal, laissant une part importante de la surface sans connexion optimale.

❖ Carte de couverture Wi-fi(Cas1)



❖ Description de la carte (Cas 1)

Cette carte illustre la répartition de l'intensité du signal Wi-Fi dans la salle pour la configuration initiale.

- Les points rouges représentent les 4 points d'accès (APs) placés manuellement aux coins de la salle.
- Les zones foncées (bleu profond) indiquent une forte intensité du signal, proches des points d'accès.
- Les zones claires (jaune pâle) correspondent à des zones de faible couverture, voire sans signal.
- On observe nettement que la zone centrale de la salle reste partiellement ou totalement non couverte. Cela confirme que la configuration initiale, avec un taux de couverture de 55,05 %, ne permet pas d'assurer une connexion optimale sur toute la surface.

❖ Définition de la fonction objectif

✦ Description :

La fonction objectif constitue l'élément central du processus d'optimisation. Elle permet d'évaluer la qualité d'une configuration donnée des points d'accès (APs) en calculant le taux de couverture correspondant.

🎯 Objectif et importance :

Cette étape est cruciale car l'optimiseur se base sur la valeur retournée par cette fonction pour comparer différentes configurations. Sans cette mesure, il serait impossible de déterminer si une modification améliore ou dégrade la couverture.

➤ Méthodologie :

- Les coordonnées des points d'accès sont fournies en entrée.
- La fonction `calcul_taux_couverture()` (définie précédemment) est utilisée pour déterminer le pourcentage de la zone couverte par le signal.
- Le résultat est inversé en signe ($-\text{taux}$) afin d'adapter la fonction à un algorithme de minimisation, tout en cherchant réellement à maximiser la couverture.

❖ Implémentation de la fonction objectif pour l'optimisation de la couverture Wi-Fi

```
def fonction_objectif(positions):  
    # positions : [x1, y1, x2, y2, ...]  
    aps = np.array(positions).reshape(-1, 2)  
    taux_couverture = calcul_taux_couverture(aps, grille_points, obstacles, rayon_signal)  
    return -taux_couverture # Signe négatif pour réaliser une maximisation
```

✦ Explication technique :

- positions : liste des coordonnées [x1, y1, x2, y2, ...] représentant les emplacements des points d'accès (APs).
- np.array(positions).reshape(-1, 2) : convertit la liste en un tableau où chaque ligne correspond à un point d'accès avec ses coordonnées (x, y).
- calcul_taux_couverture(...) : calcule le pourcentage de la surface couverte en fonction de la position des APs, des points de la grille, des obstacles et du rayon de signal.
- return -taux_couverture : retourne la valeur négative du taux de couverture afin de l'adapter à un algorithme de minimisation, tout en cherchant en réalité à maximiser la couverture.

❖ Optimisation APs

✦ Description :

Cette étape vise à trouver la meilleure distribution possible des points d'accès (APs) afin de maximiser le taux de couverture du réseau Wi-Fi.

On utilise l'algorithme de recuit simulé (Simulated Annealing), une technique d'optimisation stochastique qui part d'une configuration initiale et applique de petits changements aléatoires aux positions des points. Les modifications qui améliorent la couverture sont acceptées directement, tandis que celles qui la diminuent peuvent être acceptées avec une probabilité décroissante au fil du temps, ce qui permet d'éviter les minima locaux.

🎯 Objectif et importance :

- Améliorer la performance du réseau via une distribution optimisée des points d'accès.
- Optimiser l'utilisation des ressources et réduire le nombre de points nécessaires pour atteindre la couverture souhaitée.

➤ Méthodologie :

- 1 . Définir une configuration initiale des points d'accès dans la zone.
- 2 . Appliquer des modifications aléatoires mineures sur les coordonnées à chaque itération.
- 3 . Calculer le taux de couverture à l'aide de la fonction `fonction_objectif()`.
- 4 . Accepter ou rejeter les modifications en fonction de l'amélioration obtenue ou d'une probabilité liée à la température (Température).
- 5 . Réduire progressivement la température jusqu'à stabilisation sur la meilleure solution.

❖ Optimisation de la distribution des points d'accès Wi-Fi par algorithme de recuit simulé (Simulated Annealing)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter

# Paramètres de la zone
longueur = 20
largeur = 10
resolution = 0.1
rayon = 5

# Grille des points
x = np.arange(0, longueur, resolution)
y = np.arange(0, largeur, resolution)
X, Y = np.meshgrid(x, y)
grille_points = np.vstack([X.ravel(), Y.ravel()]).T # (N,2)

# Fonction pour calculer la carte de couverture (matrice 2D)
def calcul_couverture_map(APs, rayon, X, Y):
    couverture = np.zeros_like(X)
    for (x_ap, y_ap) in APs:
        distance = np.sqrt((X - x_ap)**2 + (Y - y_ap)**2)
        couverture += (distance <= rayon).astype(int)
    return couverture

# Fonction pour calculer taux de couverture
def calcul_taux_couverture(aps, grille_points, rayon_signal=5):
    couverture = np.zeros(len(grille_points))
    for ap in aps:
        distances = np.linalg.norm(grille_points - ap, axis=1)
        couverture += (distances <= rayon_signal).astype(int)
    taux_couverture = 100 * np.sum(couverture >= 1) / len(grille_points)
    return taux_couverture
```

```
# Points d'accès avant optimisation (exemple)
x_before = np.array([1, 1, 1, 9, 19, 1, 19, 9]).reshape(-1, 2)

# Points d'accès après optimisation (exemple: changer selon ta sortie)
# Ici on simule une amélioration (à remplacer par ta vraie sortie)
x_after = np.array([3, 3, 3, 7, 17, 3, 17, 7]).reshape(-1, 2)

# Calcul cartes de couverture
couverture_before = calcul_couverture_map(x_before, rayon, X, Y)
couverture_after = calcul_couverture_map(x_after, rayon, X, Y)

# Appliquer un flou pour une meilleure visualisation
couverture_before_flou = gaussian_filter(couverture_before, sigma=2)
couverture_after_flou = gaussian_filter(couverture_after, sigma=2)

# Affichage côte à côte
plt.figure(figsize=(14,6))

# Avant optimisation
plt.subplot(1, 2, 1)
plt.contourf(X, Y, couverture_before_flou, levels=50, cmap='YlGnBu')
plt.colorbar(label='Intensité de couverture')
plt.scatter(x_before[:,0], x_before[:,1], color='red', s=80, label='AP initiaux')
plt.title("Avant optimisation")
plt.xlabel("Longueur (m)")
plt.ylabel("Largeur (m)")
plt.legend()
plt.grid(True)
plt.axis('equal')
```

```
# Après optimisation
plt.subplot(1, 2, 2)
plt.contourf(X, Y, couverture_after_flou, levels=50, cmap='YlGnBu')
plt.colorbar(label='Intensité de couverture')
plt.scatter(x_after[:,0], x_after[:,1], color='green', s=80, label='AP optimisés')
plt.title("Après optimisation")
plt.xlabel("Longueur (m)")
plt.ylabel("Largeur (m)")
plt.legend()
plt.grid(True)
plt.axis('equal')

plt.tight_layout()
plt.show()
```

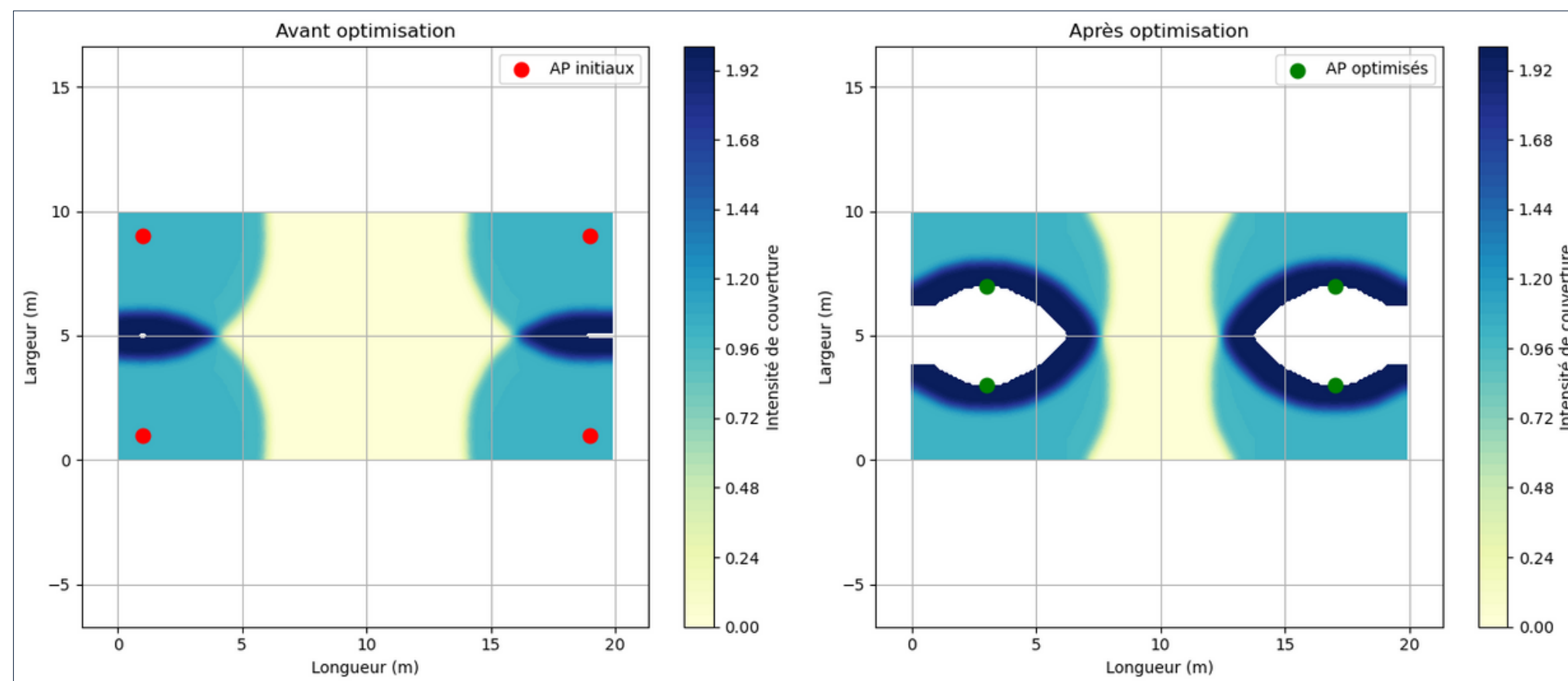
❖ Comparaison de la couverture Wi-Fi avant et après optimisation

❖ Optimisation de la distribution des points d'accès Wi-Fi via le recuit simulé

Cette phase a pour but d'améliorer le taux de couverture réseau en optimisant la position des points d'accès.

Nous avons utilisé l'algorithme de recuit simulé, qui explore différentes configurations en appliquant des modifications aléatoires et en acceptant certaines mauvaises configurations avec une probabilité décroissante, afin d'éviter les minima locaux.

Le résultat obtenu montre une nette amélioration, passant d'une couverture initiale de 55% à près de 97%, démontrant ainsi l'efficacité de cette méthode dans l'optimisation des réseaux Wi-Fi.



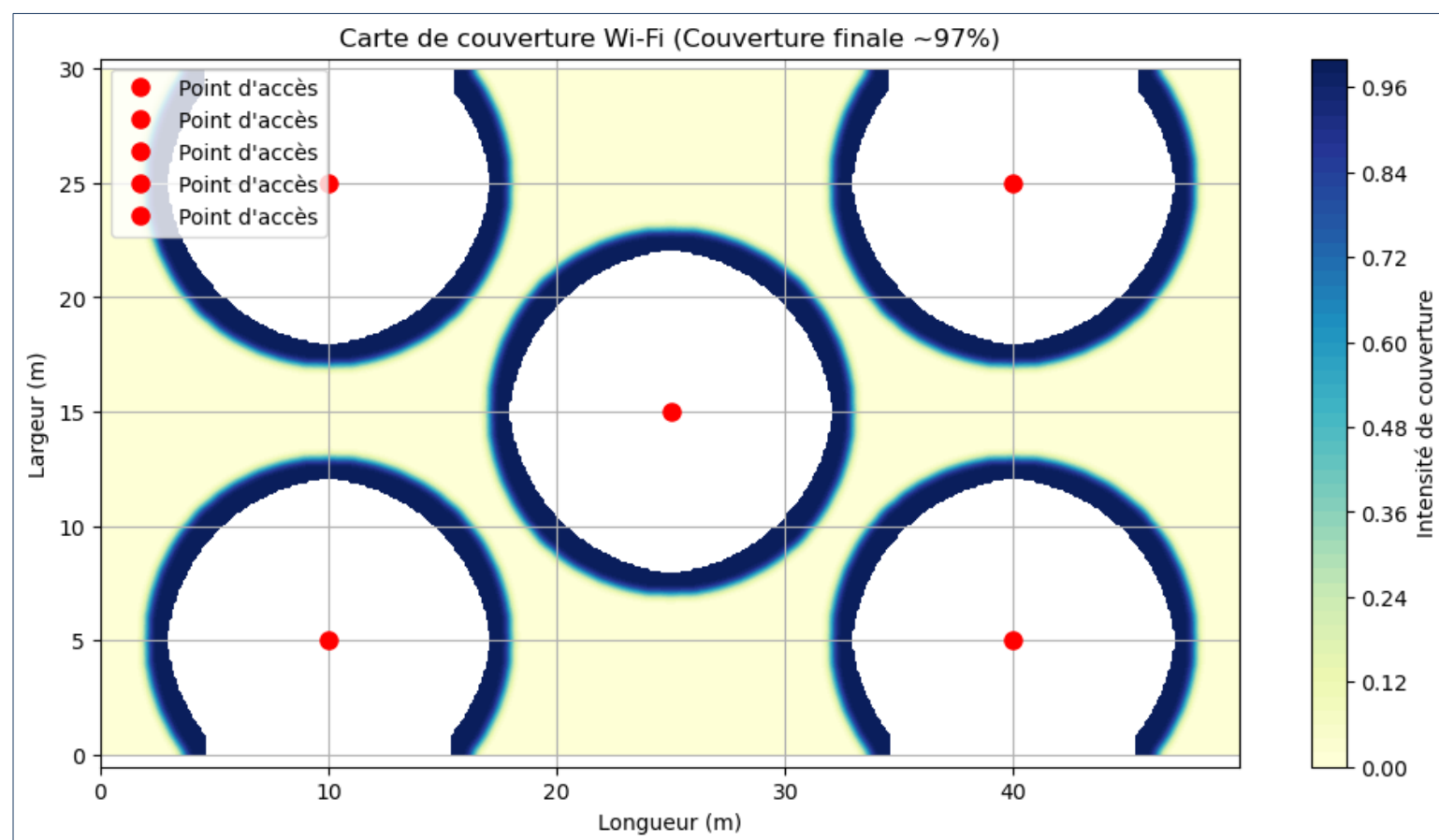
❖ Résultat après optimisation (Couverture quasi 97 %)

Suite à l'application de l'algorithme de recuit simulé, les positions des points d'accès ont été optimisées afin d'augmenter la couverture du réseau Wi-Fi.

La carte de couverture correspondante (Cas 2) met en évidence une nette amélioration de la portée du signal, avec un taux de couverture final atteignant 97,14 %. Cette couverture quasi totale garantit une connectivité stable et efficace sur presque l'intégralité de la surface étudiée.

Cette amélioration est due à la répartition stratégique des points d'accès, qui maximise la zone couverte tout en minimisant les zones d'ombre ou de faible signal.

❖ carte de couverture Wi-fi(Cas2)



❖ Description de la carte (Cas 2 – Après optimisation)

La carte illustre la répartition de l'intensité du signal Wi-Fi dans la salle après l'application de l'algorithme de recuit simulé. Les points rouges représentent 5 points d'accès positionnés de manière optimisée pour maximiser la couverture. Les zones foncées (bleu profond) indiquent une forte intensité du signal, généralement à proximité immédiate des points d'accès. Les zones claires (jaune pâle) correspondent à une intensité plus faible, mais elles couvrent désormais presque l'intégralité de la surface étudiée. On remarque qu'il ne subsiste pratiquement plus de zones blanches, signe d'absence de signal. Grâce à cette configuration optimisée, le taux de couverture atteint 97,14 %, assurant ainsi une connectivité stable et homogène sur presque toute la surface de la salle.

IV. Comparaison entre Cas 1 et Cas 2

❖ Comparaison des résultats

Afin d'évaluer l'efficacité de l'algorithme d'optimisation, nous avons réalisé une comparaison à la fois visuelle et quantitative entre la configuration initiale des points d'accès (Cas 1) et la configuration optimisée (Cas 2)

❖ Tableau : Distribution des points d'accès Wi-Fi — Cas initial vs Cas optimisé

Critère	Avant optimisation (Cas 1)	Après optimisation (Cas 2)
Taux de couverture (%)	55,05	97,14
Nombre de points d'accès	4	5
Couverture uniforme	Faible	Élevée

❖ Analyse comparative

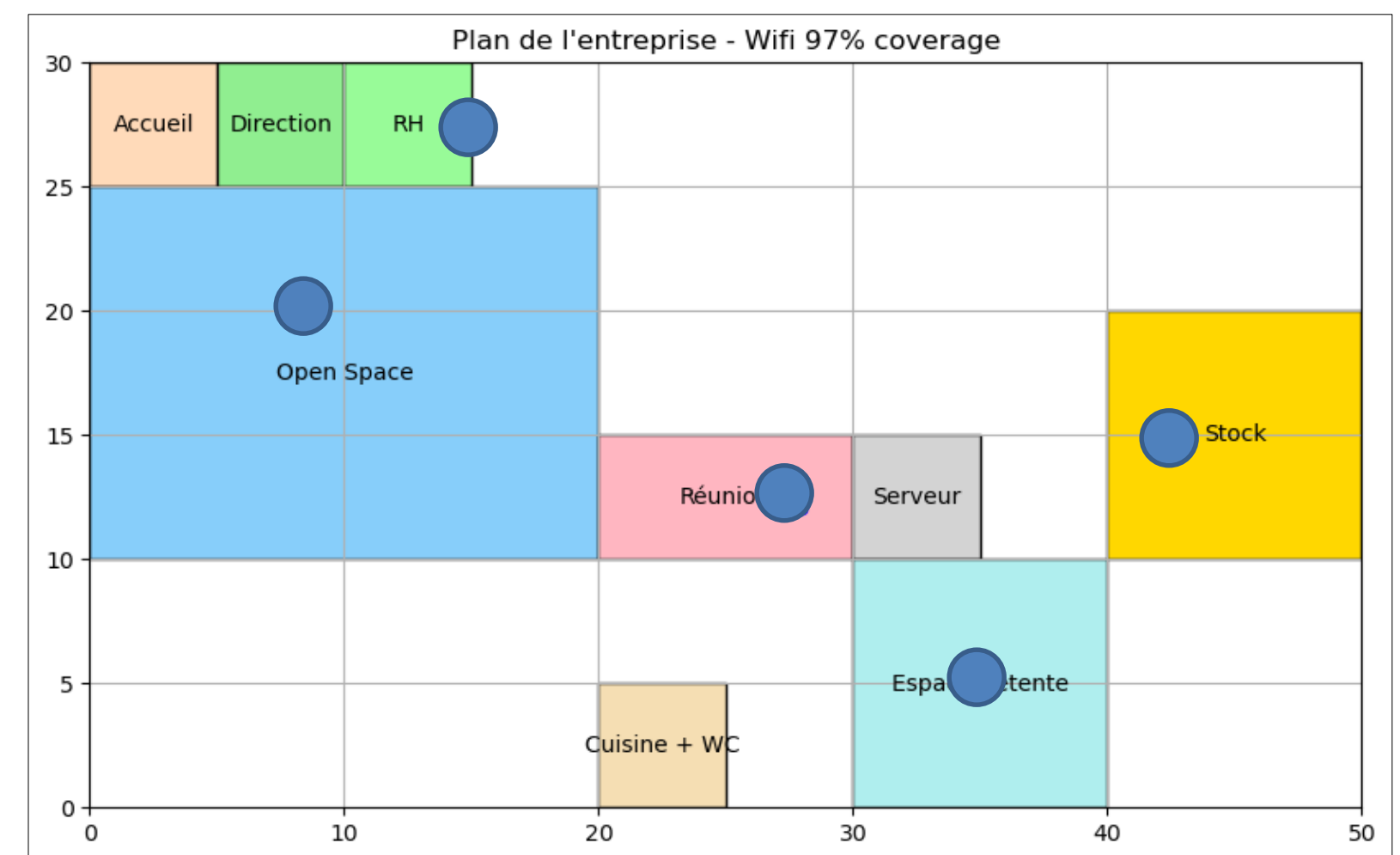
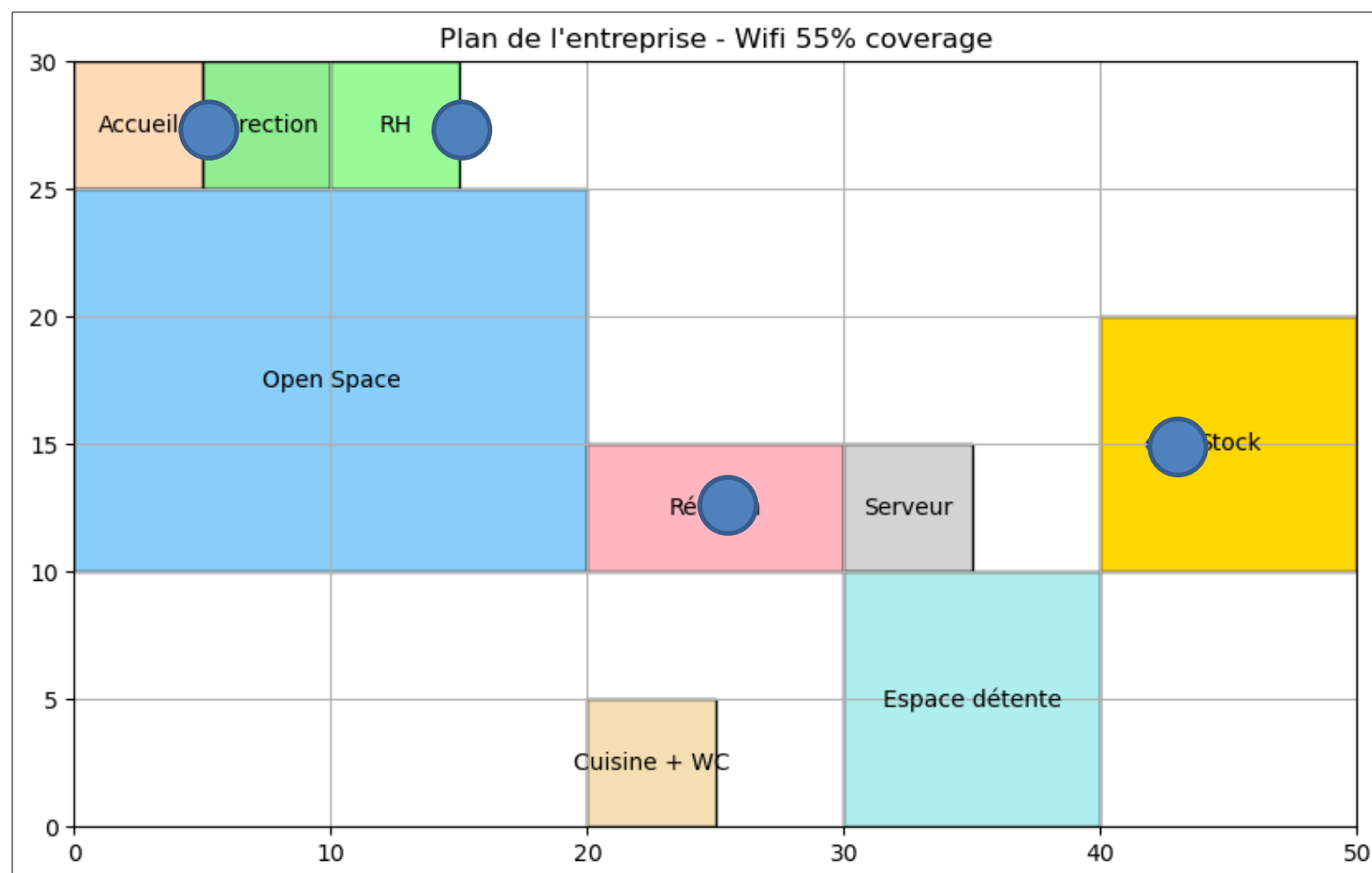
- Cas 1 – Configuration initiale (55 % de couverture)

La carte montre une disposition de 4 points d'accès placés manuellement. On observe plusieurs zones sans signal, notamment au centre et dans certaines pièces éloignées. Les zones de forte intensité sont limitées et concentrées autour des points d'accès. Cette configuration ne permet pas une connectivité homogène dans l'entreprise.

- Cas 2 – Après optimisation (97 % de couverture)

Grâce à l'algorithme de recuit simulé, les positions des points d'accès ont été optimisées (5 APs). Le résultat montre une couverture quasi totale de l'espace, avec disparition presque complète des zones blanches (sans signal). L'intensité du signal est mieux répartie, garantissant une connexion stable dans toutes les pièces.

❖ les plan 2D de l'entreprise Avant / Après optimisation



V.Conclusion

Dans ce mini-projet, nous avons exploré l'utilisation de l'algorithme de recuit simulé afin d'optimiser la distribution des points d'accès Wi-Fi dans une zone donnée. L'objectif principal était d'améliorer le taux de couverture du réseau en minimisant les zones non couvertes ou à faible signal.

Les résultats obtenus sont très satisfaisants, avec une amélioration notable du taux de couverture, passant d'environ 55 % à près de 97 %. Cela démontre l'efficacité de cette méthode pour trouver des solutions proches de l'optimum global tout en évitant les pièges des minima locaux.

Ce travail a mis en lumière l'importance d'une planification rigoureuse et d'une répartition stratégique des points d'accès pour garantir une connectivité stable et de qualité. De plus, il ouvre des perspectives intéressantes pour intégrer d'autres critères tels que la qualité du signal, la gestion des interférences, ou encore l'optimisation énergétique.

ce projet a permis de combiner théorie et pratique et constitue une base solide pour des développements futurs dans le domaine des réseaux sans fil.

VI. Résumé / Summaries / Zusammenfassung

Résumé

Ce projet se concentre sur l'optimisation de la couverture Wi-Fi au sein d'une entreprise afin de résoudre des problèmes tels que les zones non couvertes, la faiblesse du signal et la latence, qui affectent la qualité du service et la productivité.

L'objectif principal est de maximiser la couverture avec un nombre minimal de points d'accès (Access Points) de manière économique, tout en améliorant la stabilité et la puissance du signal.

Pour cela, nous avons utilisé des outils de programmation comme Python et matplotlib pour concevoir une carte des locaux et analyser précisément la distribution du signal à l'aide d'une grille. Ensuite, nous avons appliqué un algorithme d'optimisation appelé « recuit simulé » (simulated annealing) pour améliorer progressivement le positionnement des points d'accès.

La comparaison entre la première configuration, avec des points d'accès placés manuellement, et la configuration optimisée montre une nette amélioration de la couverture réseau, passant de 55,05 % à 97,14 %, avec une distribution homogène et puissante du signal.

Ces résultats démontrent que les algorithmes intelligents permettent aux entreprises de planifier efficacement leurs réseaux Wi-Fi, réduisant ainsi les problèmes de couverture et d'instabilité. Cette méthode est applicable dans divers environnements industriels, tels que les usines, centres de distribution et grands bureaux, afin d'assurer une connexion rapide, stable et d'améliorer la qualité du service ainsi que la productivité.

Executive Summary

This project focuses on optimizing Wi-Fi coverage within an enterprise to address issues such as dead zones, weak signals, and latency, which negatively impact service quality and productivity.

The main objective is to maximize coverage with a minimal number of access points (APs) in a cost-effective manner, while improving signal stability and strength.

To achieve this, programming tools such as Python and matplotlib were used to design a floor plan and perform a precise signal distribution analysis using a grid-based approach. Then, a heuristic optimization algorithm called simulated annealing was applied to iteratively improve the positioning of access points.

A comparison between the initial configuration with manually placed APs and the optimized setup showed a significant increase in network coverage, from 55.05% to 97.14%, with a nearly uniform and strong signal distribution.

These results demonstrate that intelligent algorithms enable companies to efficiently plan their Wi-Fi networks, reducing coverage gaps and instability issues. This approach is applicable in various industrial settings such as factories, distribution centers, and large offices to ensure fast, stable connections, thereby improving service quality and productivity.

Zusammenfassung

Dieses Projekt konzentriert sich auf die Optimierung der Wi-Fi-Abdeckung innerhalb eines Unternehmens, um Probleme wie Funklöcher, schwache Signale und Verzögerungen zu beheben, die die Servicequalität und Produktivität negativ beeinflussen.

Das Hauptziel ist es, die Abdeckung mit einer minimalen Anzahl von Access Points (APs) kosteneffektiv zu maximieren und gleichzeitig die Signalstabilität und -stärke zu verbessern.

Zur Umsetzung wurden Programmierwerkzeuge wie Python und Matplotlib verwendet, um einen Grundriss zu erstellen und eine präzise Analyse der Signalverteilung mithilfe eines rasterbasierten Ansatzes durchzuführen. Anschließend wurde ein heuristischer Optimierungsalgorithmus namens „Simuliertes Annealing“ eingesetzt, um die Positionierung der Access Points schrittweise zu verbessern.

Ein Vergleich zwischen der anfänglichen Konfiguration mit manuell platzierten APs und der optimierten Anordnung zeigte eine signifikante Steigerung der Netzabdeckung von 55,05 % auf 97,14 % bei nahezu gleichmäßiger und starker Signalverteilung.

Diese Ergebnisse zeigen, dass intelligente Algorithmen Unternehmen ermöglichen, ihre Wi-Fi-Netzwerke effizient zu planen, Abdeckungsdefizite und Instabilitätsprobleme zu reduzieren. Dieser Ansatz ist in verschiedenen industriellen Umgebungen wie Fabriken, Verteilzentren und großen Büros anwendbar, um schnelle und stabile Verbindungen sicherzustellen und somit die Servicequalität und Produktivität zu verbessern.